

Lamport One-Time Signature & Digital Signature Algorithm (DSA)

Information Security

BS Computer Science

BCS4A&B



Instructor: Sabghat Ullah Khan

Department of Computer Science

CUI Islamabad

Date: November 21, 2025

Contents

1	Introduction to Digital Signatures	3
1.1	What is a Digital Signature?	3
1.2	Achieving Confidentiality with Digital Signatures	3
2	Lamport One-Time Signature Scheme	3
2.1	Overview	3
3	The Three-Step Process	3
3.1	Step 1: Key Generation	3
3.2	Step 2: Signature Generation	4
3.3	Step 3: Signature Verification	5
4	Python Implementation	6
4.1	Complete Implementation with Explanation	6
5	Security Analysis	8
5.1	Why Can This Only Be Used Once?	8
6	Advantages and Disadvantages	8
6.1	Advantages	8
6.2	Disadvantages	8
7	Summary and Key Takeaways	8
7.1	Key Concepts	8
7.2	Important Formulas	9
8	Introduction to Digital Signature Algorithm (DSA)	10
8.1	What is DSA?	10
8.2	DSA vs RSA	10
8.3	DSA Components Overview	10
9	Primary Terminology	11
9.1	Key Definitions	11
10	DSA Mathematical Foundation	11
10.1	The Discrete Logarithm Problem	11
11	Steps to Perform DSA	11
11.1	Step 1: Global Public-Key Components	11
11.2	Step 2: User's Private Key Generation	12
11.3	Step 3: User's Public Key Generation	13
11.4	Step 4: Signature Generation	13
11.5	Step 5: Signature Verification	14
11.6	Why Verification Works	15

12 Python Implementation	15
12.1 Project Structure	15
12.2 Implementation 1: Miller-Rabin Primality Test	15
12.3 Implementation 2: Key Generation	17
12.4 Implementation 3: Signing	19
12.5 Implementation 4: Verification	21
13 Usage Example	24
13.1 Step-by-Step Usage	24
13.2 Testing Tampering Detection	24
14 Security Considerations	25
14.1 Critical Security Requirements	25
14.2 Attack: Reusing k	25
15 Comparison: DSA vs Lamport	26
16 Summary and Key Takeaways	26
16.1 Key Concepts	26
16.2 Important Formulas	27

1 Introduction to Digital Signatures

1.1 What is a Digital Signature?

A **digital signature** is a cryptographic technique used to verify the authenticity and integrity of a digital message or document.

Key Points

- Digital signatures provide **authentication** (proves who sent the message)
- Digital signatures provide **integrity** (proves message hasn't been altered)
- Digital signatures provide **non-repudiation** (sender cannot deny sending)
- **Important:** Digital signatures do NOT provide confidentiality by themselves

1.2 Achieving Confidentiality with Digital Signatures

To achieve confidentiality along with authentication:

1. Create the digital signature
2. Encrypt both the message AND signature using:
 - Secret key cryptography (symmetric), OR
 - Public key cryptography (asymmetric)

2 Lamport One-Time Signature Scheme

2.1 Overview

The **Lamport One-Time Signature (LOTS)** scheme is a hash-based digital signature method that:

- Uses cryptographic hash functions (like SHA-256)
- Can only be used to sign **one message** securely
- Provides quantum-resistant security

Why "One-Time"?

If the same key pair is used to sign multiple messages, attackers can forge signatures by combining parts of previous signatures.

3 The Three-Step Process

3.1 Step 1: Key Generation

Objective: Alice creates a private-public key pair.

Process:**1. Generate Private Key:**

- Use a secure random number generator
- Create **256 pairs** of random numbers
- Each number is **256 bits** long
- Total: 512 numbers (256×2)
- Store securely and NEVER share

2. Generate Public Key:

- Hash each of the 512 numbers from the private key
- This produces 512 hash values (256 bits each)
- This public key can be shared with anyone

Visual Representation:

Private Key:

Pair 0: [random_0_0, random_0_1]

Pair 1: [random_1_0, random_1_1]

...

Pair 255: [random_255_0, random_255_1]

Public Key:

Pair 0: [hash(random_0_0), hash(random_0_1)]

Pair 1: [hash(random_1_0), hash(random_1_1)]

...

Pair 255: [hash(random_255_0), hash(random_255_1)]

3.2 Step 2: Signature Generation

Objective: Alice creates a signature for her message.

Process:**1. Hash the Message:**

- Apply SHA-256 to the message
- Result: 256-bit message digest

2. Create Signature:

- For each bit in the message digest (bit 0 to bit 255):
 - If bit = 0: Select the **first** number from that pair in private key
 - If bit = 1: Select the **second** number from that pair in private key

- Result: 256 numbers forming the signature

3. Send to Bob:

- Original message
- Signature (256 numbers)

Example:

Message digest: 1 0 1 1 0 ... (256 bits)
 ↓ ↓ ↓ ↓ ↓
Select from: 1 0 1 1 0 ... (index from each pair)
 Private Key pairs

3.3 Step 3: Signature Verification

Objective: Bob verifies that Alice sent the message.

Process:

1. Hash the Received Message:

- Apply SHA-256 to the message
- Result: 256-bit message digest

2. Select Expected Hashes:

- For each bit in the message digest:
 - If bit = 0: Select **first** hash from that pair in Alice's public key
 - If bit = 1: Select **second** hash from that pair in Alice's public key
- Result: 256 expected hash values

3. Verify Signature:

- Hash each of the 256 numbers in Alice's signature
- Compare with the 256 expected hash values
- If **all match**: Signature is VALID
- If **any mismatch**: Signature is INVALID

Verification Logic:

```
For each bit i (0 to 255):
    expected_hash = public_key[bit_value][i]
    actual_hash = hash(signature[i])
    if expected_hash != actual_hash:
        return INVALID
return VALID
```

4 Python Implementation

4.1 Complete Implementation with Explanation

```

1 import hashlib
2 import secrets
3
4 # ===== KEY GENERATION =====
5 def keygen():
6     """
7     Generates a Lamport key pair (private key and public key).
8
9     Returns:
10        keypair: [private_key, public_key]
11        - private_key: 2D list [256][2] of 255-bit random numbers
12        - public_key: 2D list [256][2] of SHA-256 hashes
13    """
14    # Initialize private key structure: 2 lists of 255 elements each
15    skey = [[0] * 255, [1] * 255]
16
17    # Generate random numbers for private key
18    for i in range(len(skey)): # For both lists (0 and 1)
19        for j in range(len(skey[i])): # For each position
20            # Generate 255-bit random number
21            skey[i][j] = bin(secrets.randbits(255))[2:]
22            # Pad with zeros to ensure 255 bits
23            skey[i][j] = '0' * (255 - len(skey[i][j])) + skey[i][j]
24
25    # Initialize public key structure
26    pkey = [[0] * 255, [1] * 255]
27
28    # Generate public key by hashing private key elements
29    for i in range(len(pkey)):
30        for j in range(len(pkey[i])):
31            # Hash each private key element and store as a hex string
32            pkey[i][j] = hashlib.sha256(skey[i][j].encode()).hexdigest()
33
34    keypair = [skey, pkey]
35    return keypair

```

Listing 1: Key Generation Function

```

1 # ===== SIGNATURE GENERATION =====
2 def signgen(message, skey):
3     """
4     Generates a Lamport signature for the given message.
5
6     Args:
7         message: String message to sign
8         skey: Private key (from keygen)
9
10    Returns:
11        signature: List of 255 numbers from private key
12    """
13    # Hash the message to get 256-bit digest
14    mhash = int(hashlib.sha256(message.encode()).hexdigest(), 16)
15

```

```

16 # Initialize signature list
17 signature = [0] * 255
18
19 # For each bit position in the hash
20 for i in range(255):
21     # Extract the i-th bit using bit shifting and masking
22     k = (mhash >> i) & 1 # k is either 0 or 1
23     # Select the corresponding number from the private key
24     signature[i] = skey[k][i]
25
26 return signature

```

Listing 2: Signature Generation Function

```

1 # ===== SIGNATURE VERIFICATION =====
2 def verification(message, pkey, signature):
3     """
4     Verifies a Lamport signature against the message and public key.
5
6     Args:
7         message: Original message
8         pkey: Public key (from keygen)
9         signature: Signature to verify (from signgen)
10
11     Returns:
12         True if the signature is valid, False otherwise
13     """
14     # Hash the message to get 256-bit digest
15     mhash = int(hashlib.sha256(message.encode()).hexdigest(), 16)
16
17     # Verify each component of the signature
18     for i in range(255):
19         # Extract the i-th bit
20         k = (mhash >> i) & 1 # k is either 0 or 1
21
22         # Hash the signature component
23         verify = hashlib.sha256(signature[i].encode()).hexdigest()
24
25         # Compare with corresponding public key element
26         if pkey[k][i] != verify:
27             return False # Signature invalid
28
29     return True # All components matched, signature valid

```

Listing 3: Signature Verification Function

```

1 # ===== EXAMPLE USAGE =====
2 # Generate key pair
3 keypair = keygen()
4
5 # Message to sign
6 message = "I am god."
7
8 # Generate signature
9 signature = signgen(message, keypair[0])
10
11 # Verify signature
12 print("Signature valid:", verification(message, keypair[1], signature))

```

Listing 4: Example Usage

5 Security Analysis

5.1 Why Can This Only Be Used Once?

Scenario: Alice signs two different messages with the same key pair.

1. **Message 1 hash:** 1 0 1 1 0 ...

- Reveals: `skey[1][0]`, `skey[0][1]`, `skey[1][2]`, `skey[1][3]`, `skey[0][4]`, ...

2. **Message 2 hash:** 0 1 1 0 1 ...

- Reveals: `skey[0][0]`, `skey[1][1]`, `skey[1][2]`, `skey[0][3]`, `skey[1][4]`, ...

Problem

An attacker now knows **both** values for some pairs:

- Pair 0: Both `skey[0][0]` and `skey[1][0]` are revealed
- Pair 1: Both `skey[0][1]` and `skey[1][1]` are revealed

Attack: The attacker can forge signatures for any message whose hash bits match revealed private key elements!

6 Advantages and Disadvantages

6.1 Advantages

- **Quantum-resistant:** Secure against quantum computer attacks
- **Simple concept:** Easy to understand and implement
- **Hash-based:** Relies only on secure hash functions

6.2 Disadvantages

- **One-time use only:** New key pair needed for each message
- **Large key size:** 512×256 bits = 131,072 bits per key pair
- **Large signature size:** 256×256 bits = 65,536 bits per signature

7 Summary and Key Takeaways

7.1 Key Concepts

1. **Digital signatures** provide authentication, integrity, and non-repudiation, but NOT confidentiality

2. **Lamport signature** is a hash-based, quantum-resistant signature scheme
3. **Three steps:** Key generation $\rightarrow$ Signature generation $\rightarrow$ Signature verification
4. **Security:** Each key pair must be used only ONCE
5. **Trade-off:** High security at the cost of large key and signature sizes

7.2 Important Formulas

- **Private Key Size:** $2 \times 256 \times 256 \text{ bits} = 131,072 \text{ bits} \approx 16 \text{ KB}$
- **Public Key Size:** $2 \times 256 \times 256 \text{ bits} = 131,072 \text{ bits} \approx 16 \text{ KB}$
- **Signature Size:** $256 \times 256 \text{ bits} = 65,536 \text{ bits} \approx 8 \text{ KB}$

8 Introduction to Digital Signature Algorithm (DSA)

8.1 What is DSA?

The **Digital Signature Algorithm (DSA)** is a Federal Information Processing Standard (FIPS) for digital signatures. It is a public-key cryptographic technique explicitly used to provide digital signature functionality.

Key Characteristics of DSA

- Uses **asymmetric cryptography** (public-key cryptosystem)
- Provides **digital signature** functionality only
- **Cannot** be used for encryption or key exchange
- Based on the mathematical concept of **discrete logarithms**
- Considered **quantum-vulnerable** (unlike Lamport signatures)

8.2 DSA vs RSA

Feature	DSA	RSA
Purpose	Digital signatures only	Signatures and encryption
Mathematical basis	Discrete logarithms	Integer factorization
Signature size	Smaller (320 bits)	Larger (same as key size)
Speed	Faster signature generation	Faster verification
Random number	Required for each signature	Not required

8.3 DSA Components Overview

DSA involves the following key components:

1. **M** = Message or Plaintext
2. **H** = Hash Function (typically SHA-1 or SHA-256)
3. **PUa** = Public key of sender
4. **PRa** = Private key of sender
5. **PUG** = Global public key parameters
6. **Sig** = Signature function
7. **Ver** = Verification function

9 Primary Terminology

9.1 Key Definitions

User's Private Key (PR or x): A secret cryptographic key only possessed by the user. This key is used in digital signature generation and must be kept secure. It proves the user's genuine identity.

User's Public Key (PU or y): This key is publicly known and can be shared with anyone. It is used to verify digital signatures created with the corresponding private key.

Global Public Key (PUG): A set of parameters (p, q, g) that are shared among a group of communicating principals. These parameters define the mathematical domain for DSA operations.

Signing (Sig): The process of creating a digital signature using the user's private key. This involves mathematical operations on the message hash to generate a unique signature consisting of two components: r and s .

Verifying (Ver): The process of verifying whether a digital signature is authentic using the corresponding public key. This involves mathematical operations to compare the hash against a verification value.

10 DSA Mathematical Foundation

10.1 The Discrete Logarithm Problem

DSA security is based on the difficulty of computing discrete logarithms:

Discrete Logarithm Problem

Given: $y = g^x \mod p$

Easy: Computing y given x, g , and p

Hard: Finding x given y, g , and p (discrete logarithm)

This one-way property ensures that the private key cannot be derived from the public key.

11 Steps to Perform DSA

11.1 Step 1: Global Public-Key Components

Three parameters are generated and shared publicly among a group of users:

Parameter 1: Prime Number p

- p is a prime number with length between 512 and 1024 bits
- Must satisfy: $2^{L-1} < p < 2^L$ where $512 \leq L \leq 1024$

- L must be a multiple of 64 (increments of 64 bits)
- Condition: q divides $(p - 1)$

Mathematical notation:

$$2^{L-1} < p < 2^L \text{ where } L \in \{512, 576, 640, \dots, 1024\}$$

Parameter 2: Prime Number q

- q is an N -bit prime number
- q is a prime divisor of $(p - 1)$
- Must satisfy: $2^{N-1} < q < 2^N$ where $N = 160$ bits (typical)

Mathematical notation:

$$q \text{ is prime and } q \mid (p - 1)$$

$$2^{N-1} < q < 2^N$$

Parameter 3: Generator g

- g is selected using the formula: $g = h^{(p-1)/q} \mod p$
- h is any integer where $1 < h < (p - 1)$
- Constraint: $g > 1$
- g generates a subgroup of order q in the multiplicative group $\mathbb{Z}_p^*$

Mathematical notation:

$$g = h^{(p-1)/q} \mod p \text{ where } 1 < h < (p - 1) \text{ and } g > 1$$

Example Values

For a small example (not secure in practice):

- $p = 23$ (prime)
- $q = 11$ (prime divisor of $p - 1 = 22$)
- $h = 2$, then $g = 2^{(23-1)/11} \mod 23 = 2^2 \mod 23 = 4$

11.2 Step 2: User's Private Key Generation

The private key x is generated as follows:

- Choose x randomly or pseudorandomly
- Constraint: $0 < x < q$
- x must be kept **absolutely secret**

Mathematical notation:

$$x \in_R \{1, 2, 3, \dots, q - 1\}$$

where $\in_R$ means "randomly selected from"

11.3 Step 3: User's Public Key Generation

The public key y is computed from the private key:

- Formula: $y = g^x \mod p$
- Computing y given x is **easy** (exponentiation)
- Computing x given y is **computationally infeasible** (discrete log problem)

Mathematical notation:

$$y = g^x \mod p$$

Security Note

The security of DSA relies on the fact that given y , g , and p , it is computationally infeasible to find x . This is known as the **discrete logarithm problem**.

11.4 Step 4: Signature Generation

To sign a message M , the sender performs the following steps:

Signature Generation Process:

1. **Generate random k :**

- Choose k randomly: $0 < k < q$
- k must be unique for each signature
- k must be kept secret

2. **Compute r :**

$$r = (g^k \mod p) \mod q$$

3. **Compute hash:**

$$H(M) = \text{SHA-1 or SHA-256 hash of message}$$

4. **Compute s :**

$$s = k^{-1}[H(M) + x \cdot r] \mod q$$

where k^{-1} is the multiplicative inverse of k modulo q

5. **Signature:** The signature is the pair (r, s)

6. **Validity check:** If $r = 0$ or $s = 0$, generate a new k and repeat

Mathematical Summary:

$$k \in_R \{1, 2, \dots, q-1\}$$

$$r = (g^k \mod p) \mod q$$

$$s = k^{-1}[H(M) + x \cdot r] \mod q$$

$$\text{Signature} = (r, s)$$

Critical Security Requirement

The random value k must be:

- Generated using a cryptographically secure random number generator
- **Unique for each signature**
- **Never reused**
- Kept completely secret

Reusing k allows an attacker to recover the private key x !

11.5 Step 5: Signature Verification

To verify a signature (r', s') on message M' , the receiver performs:

Verification Process:

1. **Validity check:** Verify that $0 < r' < q$ and $0 < s' < q$

2. **Compute w:**

$$w = (s')^{-1} \mod q$$

where $(s')^{-1}$ is the multiplicative inverse of s' modulo q

3. **Compute u1:**

$$u_1 = [H(M') \cdot w] \mod q$$

4. **Compute u2:**

$$u_2 = [r' \cdot w] \mod q$$

5. **Compute v:**

$$v = [(g^{u_1} \cdot y^{u_2}) \mod p] \mod q$$

6. **Verification test:**

If $v = r'$ then signature is VALID

If $v \neq r'$ then signature is INVALID

Mathematical Summary:

$$w = (s')^{-1} \mod q$$

$$u_1 = [H(M') \cdot w] \mod q$$

$$u_2 = [r' \cdot w] \mod q$$

$$v = [(g^{u_1} \cdot y^{u_2}) \mod p] \mod q$$

$$\text{Valid} \Leftrightarrow v = r'$$

11.6 Why Verification Works

Mathematical Proof (Simplified)

If the signature is valid, then:

$$\begin{aligned}
 v &= [(g^{u_1} \cdot y^{u_2}) \bmod p] \bmod q \\
 &= [(g^{H(M)w} \cdot g^{xrw}) \bmod p] \bmod q \\
 &= [g^{(H(M)+xr)w} \bmod p] \bmod q \\
 &= [g^{(H(M)+xr) \cdot s^{-1}} \bmod p] \bmod q \\
 &= [g^k \bmod p] \bmod q \quad (\text{since } s = k^{-1}[H(M) + xr]) \\
 &= r
 \end{aligned}$$

12 Python Implementation

12.1 Project Structure

The DSA implementation consists of three main Python files:

1. `miller.py` - Contains primality testing and key generation functions
2. `keygen.py` - Generates DSA key pairs
3. `sign.py` - Signs messages
4. `verification.py` - Verifies signatures

12.2 Implementation 1: Miller-Rabin Primality Test

This module provides functions for generating large prime numbers.

```

1 # Primality Testing with the Rabin-Miller Algorithm
2 import random
3
4 def rabinMiller(num):
5     """
6     Returns True if num is a prime number using Rabin-Miller test.
7
8     Args:
9         num: Number to test for primality
10    Returns:
11        True if probably prime, False if composite
12    """
13    s = num - 1
14    t = 0
15
16    # Keep halving s while it is even
17    # Count how many times we halve s in t
18    while s % 2 == 0:
19        s = s // 2
20        t += 1
21

```



```

22     # Try to falsify num's primality 5 times
23     for trials in range(5):
24         a = random.randrange(2, num - 1)
25         v = pow(a, s, num)
26
27         if v != 1: # This test does not apply if v is 1
28             i = 0
29             while v != (num - 1):
30                 if i == t - 1:
31                     return False # num is composite
32                 else:
33                     i = i + 1
34                     v = (v ** 2) % num
35
36     return True # num is probably prime
37
38 def isPrime(num):
39     """
40     Checks whether a number is prime using trial division and the Rabin
41     -Miller test.
42
43     Args:
44         num: Number to test
45     Returns:
46         True if prime, False otherwise
47     """
48     if (num < 2):
49         return False # 0, 1, and negative numbers are not prime
50
51     # List of the first 168 prime numbers
52     lowPrimes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
53                  47,
54                  53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107,
55                  109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167,
56                  173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229,
57                  233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
58                  293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359,
59                  367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431,
60                  433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491,
61                  499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571,
62                  577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641,
63                  643, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709,
64                  719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787,
65                  797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859,
66                  863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941,
67                  947, 953, 967, 971, 977, 983, 991, 997]
68
69     # Check if num is in the list of low primes
70     if num in lowPrimes:
71         return True
72
73     # See if any of the low prime numbers can divide num
74     for prime in lowPrimes:
75         if (num % prime == 0):
76             return False
77
78     # If all else fails, call rabinMiller() to determine if num is
79     prime

```

```

77     return rabinMiller(num)
78
79 def generateLargePrime(keysize):
80     """
81     Return a random prime number of keysized bits in size.
82
83     Args:
84         keysized: Number of bits in the prime
85     Returns:
86         A prime number with keysized bits
87     """
88     while True:
89         # Generate random number in range [2^(keysized-1), 2^keysized)
90         num = random.randrange(2**(keysized-1), 2**(keysized))
91         if isPrime(num):
92             return num

```

Listing 5: miller.py - Primality Testing

12.3 Implementation 2: Key Generation

```

1 from miller import *
2 from fractions import gcd
3
4 def squareAndMultiply(x, c, n):
5     """
6     Efficient modular exponentiation using square-and-multiply
7     algorithm.
8     Computes x^c mod n.
9
10    Args:
11        x: Base
12        c: Exponent
13        n: Modulus
14    Returns:
15        Result of x^c mod n
16    """
17    z = 1
18    # Convert exponent to binary (reversed)
19    c = "{0:b}".format(c)[::-1]
20    l = len(c)
21
22    # Process each bit from most significant to least significant
23    for i in range(l-1, -1, -1):
24        z = pow(z, 2)
25        z = z % n
26        if(c[i] == '1'):
27            z = (z * x) % n
28    return z
29
30 def keyGeneration():
31     """
32     Generates DSA key pair and global parameters.
33
34     Generates:
35         - Global parameters: p, q, g
36         - Public key: h (or y)

```

```

36         - Private key: a (or x)
37
38     Saves keys to files:
39         - key.txt: Contains p, q, g, h (public parameters)
40         - secretkey.txt: Contains a (private key)
41     """
42     print("Computing key values, please wait...")
43
44     loop = True
45     while loop:
46         # Generate q: 160-bit prime
47         q = generateLargePrime(160)
48
49         # Generate k: 416-bit random number
50         k = random.randrange(2**(415), 2**(416))
51
52         # Calculate p = kq + 1
53         p = (k * q) + 1
54
55         # Ensure p is prime
56         while not (isPrime(p)):
57             k = random.randrange(2**(415), 2**(416))
58             q = generateLargePrime(160)
59             p = (k * q) + 1
60
61         # Get bit length of p
62         L = p.bit_length()
63
64         # Find generator g
65         # g = t^((p-1)/q) mod p
66         # Must satisfy: g^q mod p = 1
67         t = random.randint(1, p-1)
68         g = squareAndMultiply(t, (p-1)//q, p)
69
70         # Verify all DSA parameter constraints
71         if (L >= 512 and L <= 1024 and L % 64 == 0 and
72             (gcd(p-1, q)) > 1 and
73             squareAndMultiply(g, q, p) == 1):
74             loop = False
75
76         # Generate private key: a (random integer in range [2, q-1])
77         a = random.randint(2, q-1)
78
79         # Generate public key: h = g^a mod p
80         h = squareAndMultiply(g, a, p)
81
82         # Save public parameters to key.txt
83         file1 = open("key.txt", "w")
84         file1.write(str(p) + "\n")
85         file1.write(str(q) + "\n")
86         file1.write(str(g) + "\n")
87         file1.write(str(h))
88         file1.close()
89
90         # Save private key to secretkey.txt
91         file2 = open("secretkey.txt", "w")
92         file2.write(str(a))
93         file2.close()

```

```

94
95     print("Verification key stored at key.txt")
96     print("Secret key stored at secretkey.txt")
97
98 # Run key generation
99 keyGeneration()

```

Listing 6: keygen.py - DSA Key Generation

12.4 Implementation 3: Signing

```

1 from miller import *
2 import sys
3 import hashlib
4 import math
5
6 def computeInverse(in1, in2):
7     """
8     Computes the multiplicative inverse of in1 modulo in2.
9     Uses Extended Euclidean Algorithm.
10
11     Args:
12         in1: Number to find inverse of
13         in2: Modulus
14     Returns:
15         Multiplicative inverse of in1 mod in2
16     """
17     aL = [in1]
18     bL = [in2]
19     tL = [0]
20     t = 1
21     sL = [1]
22     s = 0
23
24     q = math.floor((aL[0] / bL[0]))
25     r = (aL[0] - (q * bL[0]))
26
27     while r > 0:
28         temp = (tL[0] - (q * bL[0]))
29         tL[0] = t
30         t = temp
31
32         temp = (sL[0] - (q * s))
33         sL[0] = s
34         s = temp
35
36         aL[0] = bL[0]
37         bL[0] = r
38
39         q = math.floor(aL[0] / bL[0])
40         r = (aL[0] - (q * bL[0]))
41
42     r = bL[0]
43     inverse = s % in2
44     return inverse
45
46 def squareAndMultiply(x, c, n):

```

```

47     """
48     Efficient modular exponentiation.
49     """
50     z = 1
51     c = "{0:b}".format(c)[::-1]
52     l = len(c)
53
54     for i in range(l-1, -1, -1):
55         z = pow(z, 2)
56         z = z % n
57         if(c[i] == '1'):
58             z = (z * x) % n
59     return z
60
61 def shaHash(fileName):
62     """
63     Computes SHA-1 hash of a file.
64
65     Args:
66         fileName: Path to file
67     Returns:
68         Integer value of hash
69     """
70     BLOCKSIZE = 65536
71     hasher = hashlib.sha1()
72
73     with open(fileName, 'rb') as afile:
74         buf = afile.read(BLOCKSIZE)
75         while len(buf) > 0:
76             hasher.update(buf)
77             buf = afile.read(BLOCKSIZE)
78
79     hex_digest = "0x" + hasher.hexdigest()
80     return int(hex_digest, 0)
81
82 def sign():
83     """
84     Signs a file using DSA.
85
86     Usage: python sign.py filename
87
88     Generates signature (c1, c2) where:
89          $c1 = r = (g^k \bmod p) \bmod q$ 
90          $c2 = s = k^{-1}[H(M) + a*r] \bmod q$ 
91     """
92     if(len(sys.argv) < 2):
93         print("Format: python sign.py filename")
94     elif(len(sys.argv) == 2):
95         print("Signing the file...")
96         fileName = sys.argv[1]
97
98         # Read public parameters
99         file1 = open("key.txt", "r")
100         p = int(file1.readline().rstrip())
101         q = int(file1.readline().rstrip())
102         g = int(file1.readline().rstrip())
103         h = int(file1.readline().rstrip())
104         file1.close()

```

```

105
106     # Read private key
107     file2 = open("secretkey.txt", "r")
108     a = int(file2.readline().rstrip())
109     file2.close()
110
111     loop = True
112     while loop:
113         # Generate random k in range [1, q-1]
114         r = random.randint(1, q-1)
115
116         # Compute c1 = r = (g^k mod p) mod q
117         c1 = squareAndMultiply(g, r, p)
118         c1 = c1 % q
119
120         # Compute c2 = s = k^(-1)[H(M) + a*r] mod q
121         c2 = shaHash(fileName) + (a * c1)
122         Rinverse = computeInverse(r, q)
123         c2 = (c2 * Rinverse) % q
124
125         # Ensure r and s are not zero
126         if (c1 != 0 and c2 != 0):
127             loop = False
128
129     # Save signature to file
130     file = open("signature.txt", "w")
131     file.write(str(c1) + "\n")
132     file.write(str(c2))
133     file.close()
134
135     print("Signature stored at signature.txt")
136
137 # Run signing
138 sign()

```

Listing 7: sign.py - Message Signing

12.5 Implementation 4: Verification

```

1 from miller import *
2 import sys
3 import hashlib
4 import math
5
6 def computeInverse(in1, in2):
7     """
8     Computes multiplicative inverse using Extended Euclidean Algorithm.
9     """
10    aL = [in1]
11    bL = [in2]
12    tL = [0]
13    t = 1
14    sL = [1]
15    s = 0
16
17    q = math.floor((aL[0] / bL[0]))
18    r = (aL[0] - (q * bL[0]))

```

```

19
20     while r > 0:
21         temp = (tL[0] - (q * bL[0]))
22         tL[0] = t
23         t = temp
24
25         temp = (sL[0] - (q * s))
26         sL[0] = s
27         s = temp
28
29         aL[0] = bL[0]
30         bL[0] = r
31
32         q = math.floor(aL[0] / bL[0])
33         r = (aL[0] - (q * bL[0]))
34
35     r = bL[0]
36     inverse = s % in2
37     return inverse
38
39 def squareAndMultiply(x, c, n):
40     """
41     Efficient modular exponentiation.
42     """
43     z = 1
44     c = "{0:b}".format(c)[::-1]
45     l = len(c)
46
47     for i in range(l-1, -1, -1):
48         z = pow(z, 2)
49         z = z % n
50         if(c[i] == '1'):
51             z = (z * x) % n
52     return z
53
54 def shaHash(fileName):
55     """
56     Computes SHA-1 hash of file.
57     """
58     BLOCKSIZE = 65536
59     hasher = hashlib.sha1()
60
61     with open(fileName, 'rb') as afile:
62         buf = afile.read(BLOCKSIZE)
63         while len(buf) > 0:
64             hasher.update(buf)
65             buf = afile.read(BLOCKSIZE)
66
67     hex_digest = "0x" + hasher.hexdigest()
68     return int(hex_digest, 0)
69
70 def verification():
71     """
72     Verifies a DSA signature.
73
74     Usage: python verification.py filename
75
76     Verification process:

```

```

77     1. Compute  $w = s^{-1} \bmod q$ 
78     2. Compute  $u_1 = [H(M) * w] \bmod q$ 
79     3. Compute  $u_2 = [r * w] \bmod q$ 
80     4. Compute  $v = [(g^{u_1} * h^{u_2}) \bmod p] \bmod q$ 
81     5. Check if  $v = r$ 
82     """
83     if(len(sys.argv) < 2):
84         print("Format: python verification.py filename")
85     elif(len(sys.argv) == 2):
86         print("Checking the signature...")
87         fileName = sys.argv[1]
88
89         # Read public parameters
90         file1 = open("key.txt", "r")
91         p = int(file1.readline().rstrip())
92         q = int(file1.readline().rstrip())
93         g = int(file1.readline().rstrip())
94         h = int(file1.readline().rstrip())
95         file1.close()
96
97         # Read signature
98         file2 = open("signature.txt", "r")
99         c1 = int(file2.readline().rstrip()) # r value
100        c2 = int(file2.readline().rstrip()) # s value
101        file2.close()
102
103        # Compute hash of message
104        t1 = shaHash(fileName)
105
106        # Compute  $w = s^{-1} \bmod q$ 
107        inverseC2 = computeInverse(c2, q)
108
109        # Compute  $u_1 = [H(M) * w] \bmod q$ 
110        t1 = (t1 * inverseC2) % q
111
112        # Compute  $u_2 = [r * w] \bmod q$ 
113        t2 = computeInverse(c2, q)
114        t2 = (t2 * c1) % q
115
116        # Compute  $v = [(g^{u_1} * h^{u_2}) \bmod p] \bmod q$ 
117        valid1 = squareAndMultiply(g, t1, p)
118        valid2 = squareAndMultiply(h, t2, p)
119        valid = ((valid1 * valid2) % p) % q
120
121        # Compare v with r
122        if(valid == c1):
123            print("Valid signature")
124        else:
125            print("Invalid signature")
126
127        # Run verification
128        verification()

```

Listing 8: verification.py - Signature Verification

13 Usage Example

13.1 Step-by-Step Usage

1. Generate Keys:

```
python keygen.py
```

Output: Creates `key.txt` and `secretkey.txt`

2. Create a Message File:

```
echo "This is a secret message" > message.txt
```

3. Sign the Message:

```
python sign.py message.txt
```

Output: Creates `signature.txt`

4. Verify the Signature:

```
python verification.py message.txt
```

Output: "Valid signature" or "Invalid signature"

13.2 Testing Tampering Detection

To demonstrate that DSA detects message tampering:

```
1 # 1. Sign original message
2 python sign.py message.txt
3
4 # 2. Verify original - should succeed
5 python verification.py message.txt
6 # Output: Valid signature
7
8 # 3. Modify the message
9 echo "This is a MODIFIED message" > message.txt
10
11 # 4. Verify modified message - should fail
12 python verification.py message.txt
13 # Output: Invalid signature
```

Listing 9: Testing Message Integrity

14 Security Considerations

14.1 Critical Security Requirements

1. Random Number k :

- Must be cryptographically random
- Must be unique for each signature
- Must never be reused
- If k is reused, the private key can be recovered!

2. Private Key Protection:

- Store `secretkey.txt` securely
- Use appropriate file permissions (e.g., `chmod 600` on Unix)
- Never transmit private key over insecure channels

3. Hash Function:

- Use SHA-256 instead of SHA-1 (SHA-1 is deprecated)
- Ensure collision resistance

4. Parameter Size:

- Modern standards recommend:
- p : 2048 or 3072 bits
- q : 224 or 256 bits
- The implementation uses 576-bit p (outdated for production)

14.2 Attack: Reusing k

Why k Reuse is Catastrophic

If k is reused for two messages M_1 and M_2 :

Two signatures:

$$\begin{aligned} s_1 &= k^{-1}[H(M_1) + x \cdot r] \mod q \\ s_2 &= k^{-1}[H(M_2) + x \cdot r] \mod q \end{aligned}$$

Attacker can compute:

$$\begin{aligned} s_1 - s_2 &= k^{-1}[H(M_1) - H(M_2)] \mod q \\ k &= \frac{H(M_1) - H(M_2)}{s_1 - s_2} \mod q \end{aligned}$$

Then recover private key:

$$x = \frac{s \cdot k - H(M)}{r} \mod q$$

15 Comparison: DSA vs Lamport

Feature	DSA	Lamport
Security basis	Discrete logarithm problem	Hash function security
Quantum resistance	Vulnerable	Resistant
Key reuse	Can sign multiple messages	One-time use only
Key size	Small (2048-bit p, 256-bit q)	Very large (131 KB)
Signature size	Small (320-512 bits)	Large (65 KB)
Computation	Requires modular exponentiation	Only hash operations
Random number	Required for each signature	Required for key generation
Maturity	Standardized (FIPS 186)	Research/niche use

16 Summary and Key Takeaways

16.1 Key Concepts

1. **DSA is a digital signature standard** based on discrete logarithms
2. **Three-step process:**
 - Key generation: Create global parameters (p, q, g) and key pair (x, y)
 - Signing: Generate signature (r, s) using private key and random k
 - Verification: Check if signature is valid using public key
3. **Security relies on:**
 - Difficulty of discrete logarithm problem
 - Cryptographically secure random number generation
 - Never reusing k values
4. **DSA provides:**
 - Authentication (proves sender identity)
 - Integrity (detects message tampering)
 - Non-repudiation (sender cannot deny)
5. **DSA limitations:**
 - Cannot encrypt (signatures only)
 - Vulnerable to quantum computers
 - Requires careful random number generation

16.2 Important Formulas

Key Generation:

$$\begin{aligned}q &: 160\text{-bit prime} \\p &= kq + 1 : 512\text{-}1024 \text{ bit prime} \\g &= h^{(p-1)/q} \pmod{p} \\x &\in \{1, 2, \dots, q-1\} \text{ (private key)} \\y &= g^x \pmod{p} \text{ (public key)}\end{aligned}$$

Signing:

$$\begin{aligned}k &\in \{1, 2, \dots, q-1\} \text{ (random, unique)} \\r &= (g^k \pmod{p}) \pmod{q} \\s &= k^{-1}[H(M) + x \cdot r] \pmod{q}\end{aligned}$$

Verification:

$$\begin{aligned}w &= (s')^{-1} \pmod{q} \\u_1 &= [H(M') \cdot w] \pmod{q} \\u_2 &= [r' \cdot w] \pmod{q} \\v &= [(g^{u_1} \cdot y^{u_2}) \pmod{p}] \pmod{q} \\ \text{Valid} &\Leftrightarrow v = r'\end{aligned}$$

End of Lecture